School of Computing

GAME PROJECT REPORT

BRIDGES Puzzle (Hashiwokakero)

TEAM – 4

Team Members :

Devara Kishore Reddy - CB.SC.U4CSE24010

Santhosh.V - CB.SC.U4CSE24047

Tulasi Sadhwika - CB.SC.U4CSE24054

B.Hasini - CB.SC.U4CSE24068

Game Mechanics / Design

The game implemented is **Hashiwokakero (Bridges Puzzle)**, a logic-based puzzle where islands must be connected using bridges while satisfying specific constraints.

Core Game Rules:

- Each island has a number indicating the required number of bridges
- Bridges can only be placed **horizontally or vertically**
- No diagonal bridges are allowed
- Bridges **cannot cross each other**
- Maximum **two bridges(0 to 2)** can exist between the same pair of islands.
- The puzzle is solved only when:
    - All islands are connected
    - Each island satisfies its required bridge count
    -

Game Design:

The game is designed as a Human vs CPU experience. Implemented using Java Swing, JPanel. The human player adds or removes bridges using simple mouse drag actions, while the CPU automatically responds after each human move. Different colours are used to clearly distinguish human bridges and CPU bridges, making gameplay intuitive and visually clear.

The game ends successfully when all islands have the required number of bridges and the entire structure forms one connected network.

Graph Model & Its Appropriateness :

- The Bridges puzzle is modeled as a graph problem:

Graph Representation:

- **Vertices (Nodes):** Islands
- **Edges:** Bridges
- **Degree of vertex:** Required number on island
- **Why Graph Model is Appropriate:**
- Connectivity can be verified using **Breadth First Search (BFS)**
- Degree constraints directly map to graph theory concepts
- Connected components help CPU decision-making
- Ensures logical correctness beyond visual completion
- Using a graph model allows:
- Efficient traversal
- Clear rule enforcement
- Reliable solution validation

Sorting Algorithm & Greedy Strategy Used:

- **Priority Queue (Min Heap)**
- Moves are sorted based on a **heuristic score**
- It ensures Efficient selection of best move
- No need to sort entire move lists repeatedly
- **Greedy Strategy:**
- The CPU follows a **Greedy approach**, making the locally best decision at each step.
- Key strategies used:

Boruvka's Principle: Prefer edges that connect two different connected components.

- It improves global connectivity early
- It prevents isolated island clusters
- It reduces the risk of creating dead-ends later in the game

Constraint Density Heuristic: It prioritizes islands with fewer remaining bridge slots.

Analysis of Strategy :

Advantages:

- Ensures early global connectivity
- Avoids isolated island clusters
- Reduces dead-end states
- Fast execution suitable for real-time gameplay

Limitations:

- Greedy strategy does not guarantee optimal solution in all cases
- Rare edge cases may lead to suboptimal decisions

Why It Works Well:

- Problem size is small
- Constraints are strict
- Deterministic behavior improves reliability

Individual Contribution Information :

Tulasi Sadhwika - CB.SC.U4CSE24054

- Designed overall game architecture
- Implemented puzzle generation logic
- Managed Java Swing UI components

Santhosh.V - CB.SC.U4CSE24047

Implemented graph algorithms (BFS, connected components)

- Developed solution verification logic
- Assisted in debugging and testing

B.Hasini - CB.SC.U4CSE24068

- Designed and implemented AI logic
- Applied Borůvka's principle and heuristic sorting
- Optimized greedy decision-making strategy

Devara Kishore Reddy – CB.SC.U4CSE24010

- Implemented input handling and rendering logic
- Managed mouse interactions and animations
- Prepared documentation and presentation material

THANK YOU